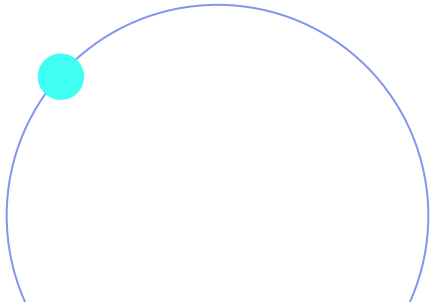
educación IT
CODE YOUR FUTURE

Datos y objetos

Lenguaje orientado a objetos

Para comenzar, es importante recordar un principio esencial: **en Python, absolutamente todo es un objeto**, es decir que cada elemento del lenguaje (números, cadenas, funciones, clases) es **una instancia de una clase con sus propios atributos y métodos**, pudiendo ser manipulado, almacenado y procesado de manera uniforme y dinámica. Esta característica define profundamente la filosofía del lenguaje y su forma de gestionar la información.

Esta concepción de todo como objeto permite una programación más intuitiva, donde **cada elemento tiene capacidades inherentes más allá de su valor básico**. Los programadores pueden interactuar con los datos de manera más rica y expresiva, aprovechando los métodos y propiedades de cada tipo de objeto.



Python: lenguaje tipado

Python es un **lenguaje de "tipado" dinámico**. Un lenguaje de programación es dinámicamente tipado si **una variable puede tomar valores de distinto tipo**. Esto quiere decir que las variables no son una suerte de cajas que aceptan un único tipo de datos, sino más bien etiquetas que se le asignan a un objeto de cualquier tipo. Los tipos de datos permiten **clasificar la información** que contiene una variable.

No es necesario ahondar en cómo el intérprete maneja internamente las variables, simplemente tener en cuenta que no están ligadas a un tipo de dato en específico, y que se llaman **objetos.**



Tipos de datos

Los tipos de datos básicos en Python incluyen:

Enteros (`int`)	Representan números sin decimales, como 5 o -3.
Flotantes (`float`)	Manejan números con punto decimal (racionales), permitiendo representaciones como 3.14 o -0.5.
Cadenas (`str`)	Son secuencias de caracteres que permiten almacenar texto, siempre encerradas entre comillas simples o dobles.
Booleanos (`bool`)	Representan los valores lógicos verdadero o falso.
Datos complejos (`complex`)	Listas, tuplas, diccionarios y conjuntos, cada uno con características y comportamientos específicos.



A los datos podemos asignarles el nombre que queramos, respetando **no usar las palabras reservadas** (lista de palabras reservadas) de Python **ni espacios, guiones o números al principio**.

Hagamos una prueba

Comencemos por abrir la consola interactiva de Python y escribir lo siguiente:

```
>>> dato = 1
```

Hemos creado un objeto que contiene el número 1. Escribiendo el nombre de un objeto en la consola interactiva, obtenemos su valor.

```
>>> dato
1
```

Los caracteres **>>>** indican que estamos trabajando con una consola interactiva.



Función *type()*

La **función `type()`**, nos permite averiguar el tipo de dato de una variable.

```
>>> type(dato)
<class ‘int’>
```

Usando la instrucción `type`, podemos saber en tiempo de ejecución **a qué tipo de dato corresponde el contenido de una variable**. Lo que nos interesa es lo que aparece **entre comillas simples** luego de cada llamada a `type()`, que es cómo Python nomina a los tipos de datos.

```
>>> type(a)
<class 'int'>

>>> type(saludo)
<class 'str'>

>>> type(pi)
<class 'float'>

>>> type(encendido)
<class 'bool'>

>>> type(c)
<class 'complex'>
```

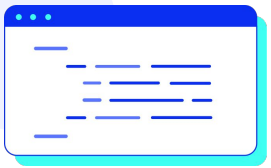


Una misma variable puede contener distintos tipos de dato durante la ejecución de un mismo programa.

None

Cuando queremos crear un objeto, pero por el momento **no asignarle ningún valor**, generalmente se utiliza **`None`** (en inglés, literalmente, “nada”). Esta palabra se usa para que la variable (u objeto) no tenga un tipo de dato asociado. `None` no es un tipo de dato.

```
>>> a = None
>>> type(a)
<class 'NoneType'>
```



educación IT
CODE YOUR FUTURE

Expresiones y variables

Expresiones

El código **7 + 5** es una expresión. Una **expresión** es un **fragmento de código que retorna un resultado**.

Por **retornar** nos referimos a que el lenguaje reemplazará ese fragmento de código por su resultado. Así, en todas las partes del código en donde aparezca 7 + 5, por cuanto se trata de una expresión, el lenguaje lo reemplazará por 12.

El código:

```
print(7 + 5)
```

Es similar a:

```
print(12)
```



El término “reemplazar” es usado aquí con fines didácticos para comprender el concepto de expresión. En realidad, el proceso de evaluación de las expresiones que aparecen en un código es más complejo.

Variables

Ahora que sabemos lo que es una expresión, el concepto que aquí nos ocupa se torna más sencillo. Una **variable** es el nombre que se le otorga a una expresión.

La **sintaxis** para crear una variable es:

```
variable = expresión
```

Donde la variable es un **nombre cualquiera** (escrito, por convención, siempre en minúscula) y la expresión es **otra expresión cualquiera** que queramos nombrar.

Por ejemplo:

```
a = 7 + 5
```

Luego de la creación de una variable (cuya operación se denomina **definición**), podemos usarla tantas veces como consideremos necesario en todo código subsiguiente. Incluso para **crear otras expresiones y otras variables**:

```
a = 7 + 5
print(a)
```

```
a = 7 + 5
print(a + 10)
```

```
a = 7 + 5
b = a + 10
print(b)
```

La **definición** de una variable no es una expresión, pues no retorna ningún resultado, sino una sentencia.



Otros ejemplos

Las variables son en sí mismas expresiones, por lo que a ellas también se le aplica lo que dijimos respecto de estas últimas: siguiendo el último ejemplo, Python reemplazará **a** por **12** en toda ocurrencia de **a** y **saludo** por **"Hola mundo"** en toda ocurrencia de saludo.

```
a = 12
saludo = "Hola mundo"
print(a)
print(saludo)
```

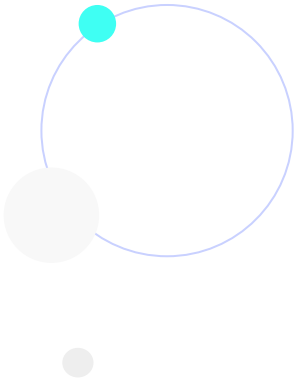


Observa este ejemplo:

```
# Declaración de variables
nombre = "Ana"
edad = 25
altura = 1.68

# Expresiones
edad_en_5_anios = edad + 5  # Sumar 5 a la edad actual
es_mayor_de_edad = edad >= 18  # Comparación para verificar si es mayor de edad
mensaje = f"Hola, {nombre}. Tienes {edad} años y mides {altura} metros."  # Concatenación

# Imprimir resultados
print(mensaje)
print("En 5 años tendrás:", edad_en_5_anios)
print("¿Eres mayor de edad?", es_mayor_de_edad)
```



El resultado será:

```
Hola, Ana. Tienes 25 años y mides 1.68 metros.
En 5 años tendrás: 30
¿Eres mayor de edad? True
```

Tipos de variables

Variables numéricas

Existen dos categorías principales de variables numéricas:

Números enteros (*integers*)	Números de punto flotante (*float*)
Son valores sin parte decimal, pudiendo ser positivos, negativos o cero, y se utilizan comúnmente para contadores, índices y operaciones que requieren valores discretos.	Permiten representar valores con parte decimal, siendo esenciales para cálculos que demandan mayor precisión matemática.

La declaración de la variable implica **asignar un nombre significativo y un valor inicial**, lo que permite su posterior utilización en diversas operaciones como suma, resta, multiplicación, división y potenciación.

En este ejemplo puedes ver variables numéricas:

```python
# Variable integer para representar la cantidad de estudiantes
num_estudiantes = 25

# Variable float para representar el promedio de calificaciones
promedio_calificaciones = 8.75

# Operaciones con variables integer
total_estudiantes = num_estudiantes * 2
aumento_estudiantes = num_estudiantes + 5

# Operaciones con variables float
promedio_ajustado = promedio_calificaciones + 0.5
porcentaje_aprobacion = promedio_calificaciones / 10 * 100

# Combinando tipos de variables
costo_por_estudiante = 125.50
gasto_total = num_estudiantes * costo_por_estudiante

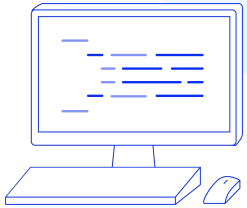
# Impresión de resultados
print(f"Número de estudiantes: {num_estudiantes}")
print(f"Promedio de calificaciones: {promedio_calificaciones}")
print(f"Total de estudiantes: {total_estudiantes}")
print(f"Gasto total: {gasto_total}")
```

Variables *string*

Las variables *string*, también conocidas como variables de texto o cadenas de caracteres, representan **secuencias de caracteres** que pueden contener letras, números, símbolos y espacios, lo que permite trabajar con **cualquier tipo de texto**.

Las variables *string* se definen utilizando **comillas simples o dobles**.

Estas variables pueden ser concatenadas, divididas, transformadas y analizadas mediante diversos métodos. Su **versatilidad** permite realizar operaciones como convertir texto a mayúsculas o minúsculas, buscar subcadenas, reemplazar caracteres y realizar múltiples transformaciones que son esenciales en el desarrollo de *software*.



Aquí tienes un ejemplo que ilustra el uso de *`string`*:

```
# Declaración de variables string
nombre = "María"
apellido = 'González'
ciudad = "Barcelona"

# Concatenación de strings
nombre_completo = nombre + " " + apellido

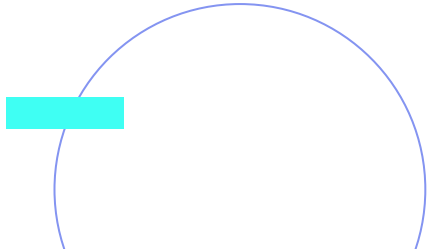
# Métodos de transformación
nombre_mayusculas = nombre.upper()
nombre_minusculas = nombre.lower()

# Longitud de la cadena
longitud_nombre = len(nombre)

# Substring
inicial_nombre = nombre[0]

# Interpolación de strings
presentacion = f"Hola, me llamo {nombre_completo} y vivo en {ciudad}"
```

```
# Impresión de resultados
print(nombre_completo)
print(nombre_mayusculas)
print(nombre_minusculas)
print(f"Longitud del nombre: {longitud_nombre}")
print(f"Inicial del nombre: {inicial_nombre}")
print(presentacion)
```



Variables booleanas

Las variables booleanas representan **valores lógicos de verdad**, limitándose únicamente a dos posibles estados: **verdadero (*True*)** o **falso (*False*).**

Estas variables son esenciales para la toma de decisiones, el control de flujo y la implementación de estructuras condicionales en cualquier lenguaje de programación. Su importancia radica en la **capacidad de evaluar condiciones**, pues permite que los programas tomen decisiones basadas en diferentes escenarios.

Las variables booleanas se utilizan frecuentemente en **estructuras de control** como sentencias `if`, bucles, operaciones de comparación y lógica de programación, ya que **facilitan la creación de algoritmos** que pueden responder de manera dinámica a diferentes situaciones.



Este es un ejemplo que muestra el uso de variables booleanas:

```python
# Declaración de variables booleanas
esta_lloviendo = True
tiene_paraguas = False
es_fin_de_semana = True

# Operaciones lógicas
puede_salir = not esta_lloviendo and tiene_paraguas

# Comparaciones que generan valores booleanos
edad = 20
es_mayor_de_edad = edad >= 18

# Combinación de condiciones
puede_votar = es_mayor_de_edad and es_fin_de_semana

# Impresión de resultados
print(f"¿Está lloviendo? {esta_lloviendo}")
print(f"¿Tiene paraguas? {tiene_paraguas}")
print(f"¿Puede salir? {puede_salir}")
print(f"¿Es mayor de edad? {es_mayor_de_edad}")
print(f"¿Puede votar? {puede_votar}")
```

Comentarios

Utilidad de los comentarios

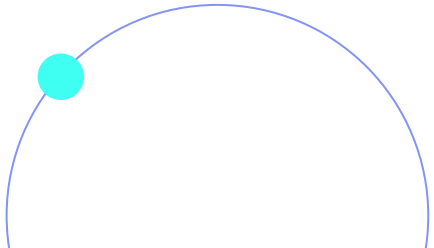
Los comentarios son líneas dentro de un archivo de código que Python ignorará por completo. Son útiles para **hacer anotaciones en el código.**

La **sintaxis** para crearlos es **#** (numeral). Por ejemplo:

```
# Imprimir un mensaje en pantalla.
print("Hola mundo")
```

Ocasionalmente, aunque poco recomendado, se pueden colocar en la misma línea que una instrucción de Python. Observa este ejemplo:

```
x = 1  # Asignar 1 a la variable x.
```



El comentario generado con **#** es de un renglón, pero existe la posibilidad de generar comentarios tipo párrafo. Los **comentarios multilínea** permiten generar comentarios de varios renglones. Se generan con triple comilla simple o doble para abrir y cerrar.

```
’’’’’’
Esto es
Un
Comentario
’’’’’’
```



Documentar el código con comentarios

Documentar un código en Python con comentarios es fundamental para **garantizar su comprensión y mantenimiento** a largo plazo.

- Los comentarios deben ser **claros, concisos y colocarse estratégicamente** para explicar partes clave del código sin saturarlo.
- Es importante usar comentarios para **describir la intención de bloques de código**, no solo lo que hacen, ya que el propósito suele ser menos evidente que la lógica.
- Comenta **únicamente lo necesario**, evitando redundancias con el propio código.
- Sigue las **convenciones de estilo de Python**.

> Por ejemplo, un comentario efectivo en una función podría describir qué parámetros espera, qué hace y qué devuelve. Esto no solo ayuda a otros programadores, sino que también mejora tu propio flujo de trabajo cuando vuelves a trabajar en el código después de un tiempo.

